

Project Proposal: UniPulse — Student Success & Academic Intelligence Platform

1. Project Title

UniPulse: A Full-Stack Student Success and Academic Intelligence Platform

2. Project Overview

UniPulse is a web-based university student success and academic intelligence platform designed to combine traditional academic management functions with advanced data analytics.

Many existing university information systems mainly focus on storing student information such as registrations, module enrollments, attendance, examination marks, and final results. Although these systems are useful for managing academic records, they often provide limited support for analyzing student progress, identifying students who may require academic support, examining performance trends, evaluating module difficulty, or supporting students in planning future academic outcomes.

UniPulse aims to address this gap by creating a unified platform that combines:

- Full-stack web application development
- Academic data management
- Student and lecturer dashboards
- Data processing and ETL
- Statistical and exploratory data analysis
- Academic performance analytics
- Academic-risk indicators
- Grade and GPA simulation
- Intervention management
- Power BI reporting
- Optional machine-learning-based prediction

The platform will support four primary user roles:

- Students
- Lecturers
- Academic Advisors
- Administrators/Department Heads

The project is intended to demonstrate both **Software Engineering** and **Data Analytics** skills within one realistic system.

3. Background

Universities generate a large amount of academic data throughout a student's degree program.

Examples include:

- Student registrations
- Program enrollments
- Module enrollments
- Attendance records
- Assignment marks
- Quiz results
- Mid-semester examination marks
- Final examination marks
- Learning-management-system activities
- Repeated modules
- Academic support interventions

However, simply storing this information does not necessarily help a university understand what is happening to its students.

For example, a student may gradually demonstrate the following pattern:

1. Attendance begins to decrease.
2. Assignment submissions become late.
3. Quiz performance starts declining.
4. Several assessments are missed.
5. Overall module performance decreases.
6. The student eventually fails the module.

In many traditional systems, this problem becomes obvious only after the final examination results are released.

UniPulse proposes a more proactive approach.

Instead of simply recording historical academic information, the system will continuously analyze academic patterns and provide meaningful information to students, lecturers, academic advisors, and university management.

For example:

Student perspective

"My Statistics performance has decreased during the last three assessments, and my attendance has fallen from 88% to 69%."

Lecturer perspective

"Twenty-three students in my Database Systems module currently demonstrate patterns that may require academic attention."

Department perspective

"Algorithm Design has consistently demonstrated one of the highest failure rates during the last three semesters."

Therefore, UniPulse transforms academic records into useful decision-support information.

4. Problem Statement

Most university academic management systems are primarily designed to store and manage academic information rather than provide proactive analytical support.

As a result, several problems may arise:

- Students may not clearly understand their academic progress until final results are released.
- Students may find it difficult to determine the marks required in remaining assessments to achieve a target grade.
- Lecturers may find it difficult to identify students whose academic performance is gradually declining.
- Academic advisors may lack a centralized mechanism for monitoring students who require support.
- Universities may have large amounts of academic data but limited mechanisms for analyzing trends across modules, semesters, programs, and cohorts.
- Difficult modules or assessments may only be identified informally.
- Academic intervention activities may be performed without properly analyzing their outcomes.
- Institutional decision-makers may not have interactive dashboards for evaluating academic performance and student engagement.

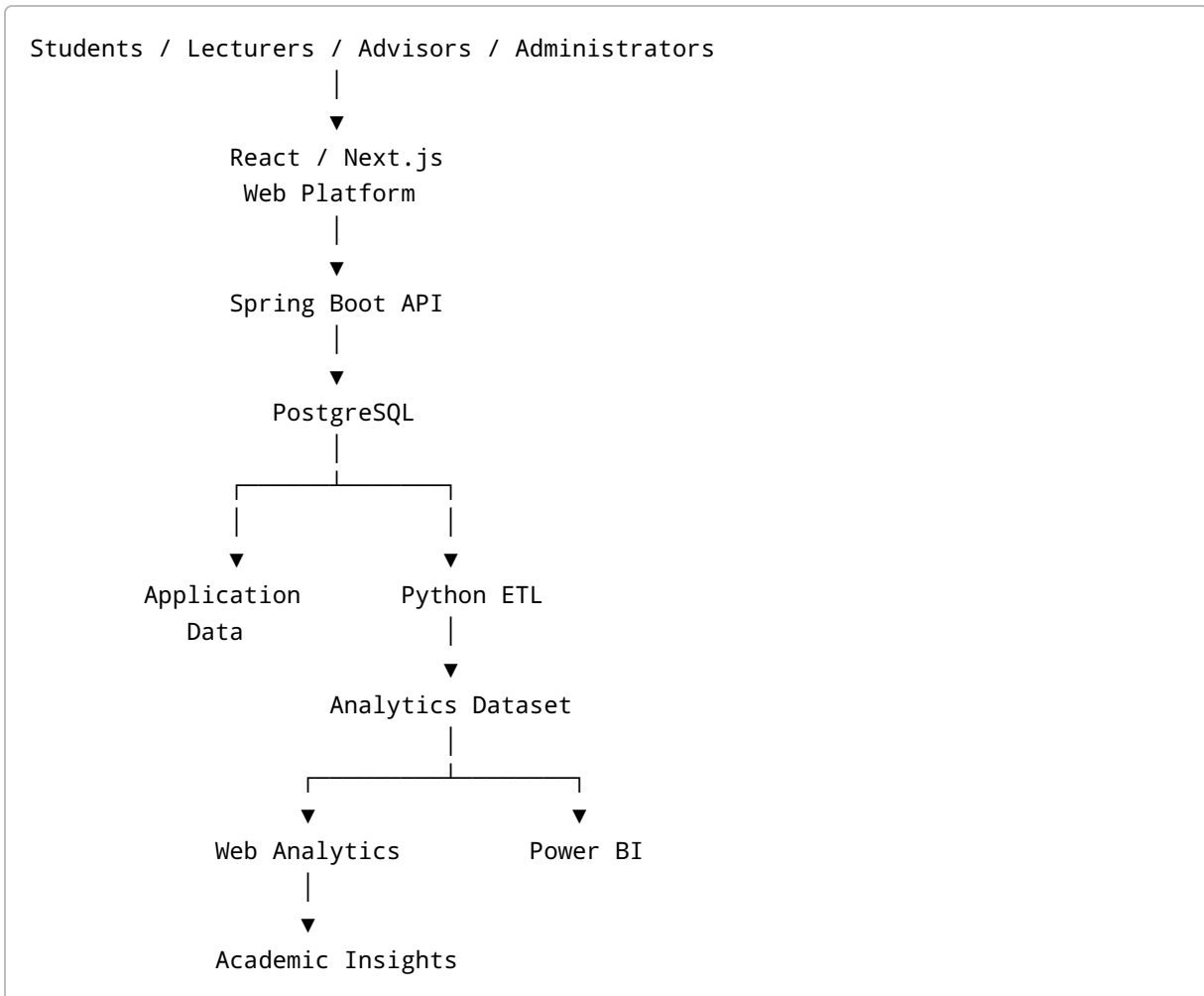
Therefore, there is a need for a platform that combines academic data management with analytics and decision-support capabilities.

5. Proposed Solution

The proposed solution is **UniPulse**, a responsive web-based academic intelligence platform.

The system will collect and manage academic information through a full-stack application and process that information using an analytics pipeline.

The high-level system will operate as follows:



The platform will combine operational university functions with analytical capabilities.

6. Main Aim

The main aim of the project is:

To design and develop a full-stack data-driven university platform that manages academic information and transforms student academic data into meaningful insights for monitoring performance, supporting academic planning, identifying potential academic difficulties, and improving institutional decision-making.

7. Project Objectives

7.1 Software Engineering Objectives

The system will:

1. Develop a secure web application with multiple user roles.
 2. Implement student, lecturer, advisor, module, program, and assessment management.
 3. Implement attendance and academic result management.
 4. Develop RESTful APIs using Spring Boot.
 5. Implement authentication and role-based authorization.
 6. Design a relational PostgreSQL database for academic information.
 7. Implement validation, exception handling, and secure API communication.
 8. Develop responsive dashboards using React/Next.js.
 9. Implement automated backend and API testing.
 10. Containerize major application components using Docker.
-

7.2 Data Analytics Objectives

The system will:

1. Develop an ETL pipeline for extracting, cleaning, transforming, and analyzing academic data.
2. Analyze student academic-performance trends.
3. Analyze attendance and assessment relationships.
4. Calculate academic KPIs.
5. Identify modules and assessments associated with comparatively poor student performance.
6. Develop academic attention indicators.
7. Analyze academic support/intervention outcomes.

8. Perform cohort and semester comparisons.
 9. Develop Power BI dashboards for institutional reporting.
 10. Experiment with predictive models for academic-risk classification as an advanced project component.
-

8. Target Users

8.1 Student

Students will use UniPulse to monitor and plan their academic progress.

Major functions include:

- View enrolled modules
 - View assessment results
 - View attendance
 - View GPA
 - View semester performance
 - View academic trends
 - Set grade goals
 - Use grade simulator
 - Use GPA planner
 - View academic health information
 - View recommendations
 - Receive academic notifications
-

8.2 Lecturer

Lecturers will manage academic activities related to their assigned modules.

Functions include:

- View assigned modules
- View enrolled students
- Create assessments
- Enter assessment marks
- Upload results
- Record attendance
- View class analytics
- View grade distributions
- Identify students requiring attention

- Create academic-support recommendations
 - Monitor intervention progress
-

8.3 Academic Advisor

Academic advisors will focus mainly on student support.

Functions include:

- View assigned students
 - View student progress summaries
 - View academic attention indicators
 - Review performance trends
 - Record academic interventions
 - Add advisor notes
 - Schedule follow-ups
 - Monitor outcomes
-

8.4 Administrator / Department Head

Administrative users will receive high-level academic information.

Functions include:

- Manage users
 - Manage faculties
 - Manage departments
 - Manage programs
 - Manage semesters
 - Manage modules
 - View institutional analytics
 - Compare programs
 - Analyze cohorts
 - Analyze module performance
 - View academic-risk statistics
 - Generate reports
 - Monitor data quality
-

9. Proposed Core Modules

9.1 Authentication and User Management Module

The authentication module will provide:

- User registration
- Login
- Logout
- Password hashing
- JWT authentication
- Role-based access control
- Profile management
- Password reset
- Account management

Roles:

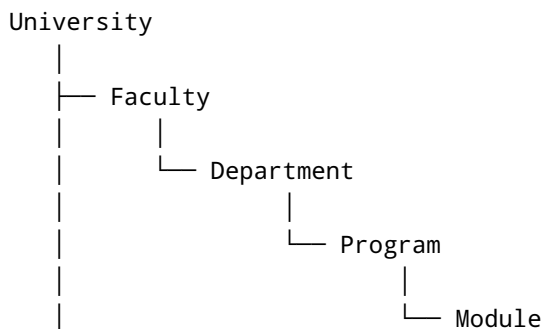
STUDENT
LECTURER
ADVISOR
ADMIN

Spring Security will be used for authentication and authorization.

10. Academic Structure Management

This module manages the university academic hierarchy.

Example structure:



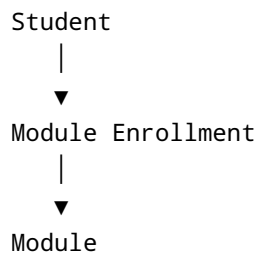
The system will manage:

- Faculties
- Departments
- Programs
- Modules
- Academic years
- Semesters

11. Module Enrollment Management

Students will be enrolled into modules.

Example:



Enrollment information may include:

- Student
- Module
- Semester
- Academic year
- Enrollment date
- Enrollment status
- Final grade

12. Assessment Management Module

Lecturers will create assessments such as:

- Assignments
- Quizzes
- Mid-semester examinations
- Practical examinations

- Projects
- Final examinations

Example:

Database Systems

Assignment 1	10%
Quiz	10%
Midterm	30%
Final Exam	50%

The system will validate that the total module assessment weight equals 100%.

13. Results Management

Lecturers will enter student results.

The system will automatically calculate:

- Weighted marks
- Module total
- Letter grades
- Pass/fail status
- Semester GPA
- Cumulative GPA

Example:

Assignment:	$75 \times 20\% = 15$
Midterm:	$68 \times 30\% = 20.4$
Final:	$72 \times 50\% = 36$
Total = 71.4	

14. Attendance Management

Lecturers can create attendance sessions.

Example:

Module:
Database Systems

Date:
2026-08-10

Session:
Lecture

Students:
142

Each student can be recorded as:

PRESENT
ABSENT
LATE
EXCUSED

The system calculates attendance percentage automatically.

15. Student Dashboard

The student's main dashboard could show:

UNIPULSE		
Current GPA	Attendance	Academic Health
3.24	82%	76/100
Semester Progress		
Database Systems	72%	
Data Structures	77%	
Statistics	61%	
Software Engineering	79%	

Performance Trend



Attention Areas

Statistics

Attendance declined from 79% to 68%.

Database Systems

Recent quiz score is below your semester average.

16. Academic Progress Twin

A major unique feature of UniPulse will be the **Academic Progress Twin**.

It represents the student's current academic condition using several indicators.

Example:

Academic Progress Twin

GPA	3.24
Attendance	82%
Assessment Average	74%
Submission Rate	91%
Engagement	69%
Credits Completed	64

Academic Health

76 / 100

Trend

Improving

The Progress Twin can be periodically recalculated as new information becomes available.

It provides a simple summary without forcing users to inspect multiple screens.

17. Academic Health Score

The platform can calculate an explainable academic health indicator.

Example formula:

Academic Health Score	
Academic Performance	40%
Attendance	20%
Submission Completion	15%
Engagement	15%
Performance Trend	10%

Example:

```
Performance = 78
Attendance = 82
Submission = 90
Engagement = 68
Trend = 70

Health =
(78 × 0.40)
+
(82 × 0.20)
+
(90 × 0.15)
+
(68 × 0.15)
+
(70 × 0.10)

= 78.3
```

Result:

Academic Health = 78 / 100

The scoring methodology will be documented and configurable.

18. What-If Grade Simulator

One of UniPulse's signature features will be a **What-If Grade Simulator**.

The student can answer questions such as:

"What mark do I need in my final examination to get a B+?"

Example:

Assignments	20%
Midterm	30%
Final	50%

Current:

Assignments	72
Midterm	61

Current contribution:

$$72 \times 0.20 = 14.4$$

$$61 \times 0.30 = 18.3$$

$$\text{Total} = 32.7$$

If B+ requires 70:

$$70 - 32.7 = 37.3$$

$$37.3 / 0.50 = 74.6$$

Therefore:

Required Final Examination Mark

75%

The platform can display:

Target Grade: B+

Required Final Mark: 75%

If final = 60 → Overall 62.7

If final = 70 → Overall 67.7

If final = 75 → Overall 70.2

If final = 80 → Overall 72.7

If final = 90 → Overall 77.7

19. GPA Goal Planner

Students will also be able to specify a target GPA.

Example:

Current projected GPA:

3.19

Target:

3.50

The platform will calculate different grade combinations that may achieve the target.

Example:

Statistics

Current projection: C+

Target scenario: B+

Database Systems

Current projection: B

Target scenario: A-

Economics

Current projection: B

Target scenario: B+

Projected GPA:

3.19 → 3.51

20. Academic Performance Analytics

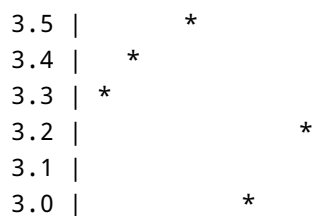
The platform will analyze:

- Student performance over time
- Module performance
- Assessment performance
- Grade distributions
- Semester comparisons
- Program performance
- Failure rates
- Pass rates
- GPA trends

Example:

Semester	GPA
Semester 1	3.31
Semester 2	3.42
Semester 3	3.06
Semester 4	3.28

Trend:



21. Attendance Analytics

The platform will analyze:

- Overall attendance

- Attendance by module
- Attendance trends
- Attendance by semester
- Relationship between attendance and academic performance

Example:

Attendance Range	Average Result
90-100%	79%
80-89%	73%
70-79%	67%
60-69%	60%
<60%	52%

The system may report:

Students with higher attendance tended to demonstrate higher assessment performance within the analyzed dataset.

The system will avoid incorrectly interpreting correlation as causation.

22. Assessment Difficulty Analyzer

UniPulse will analyze assessment-level performance.

Example:

Assessment	Average
Quiz 1	78%
Assignment 1	73%
Quiz 2	69%
Midterm	48%
Assignment 2	75%

The system can highlight:

Possible Difficult Assessment
Midterm Examination

Class average: 48%

If question/topic information is available:

Topic	Average
SQL Basics	81%
Joins	67%
Normalization	43%
Transactions	57%
Indexing	52%

The system can identify:

Lowest performing topic:

Normalization

This could help lecturers identify concepts that require additional teaching support.

23. Module Difficulty Analysis

At the department level, UniPulse can calculate a Module Difficulty Index.

Possible factors:

Failure Rate
Average Result
Repeat Rate
Withdrawal Rate
Grade Distribution

Example:

Module	Difficulty Index
Advanced Statistics	82
Algorithms	79
Database Systems	63

Economics	50
Web Development	44

The purpose is not to label a module as objectively difficult but to identify comparatively challenging academic patterns.

24. Academic Attention Indicator

UniPulse will provide an explainable academic attention indicator.

The system should avoid claiming:

"This student will fail."

Instead, it should communicate:

"Current academic patterns suggest that this student may require additional academic attention."

Initial rule-based model:

Attendance < 60%	+30
Current average < 50%	+30
Two or more missed assessments	+20
Declining performance	+10
Low engagement	+10

Maximum score:

100

Categories:

0-29	Low
30-59	Medium
60-100	High

Example:

Academic Attention

HIGH

Main signals:

Attendance	High impact
Assessment average	High impact
Missed assessments	Medium impact
Recent trend	Medium impact

This provides transparency.

25. Predictive Analytics Extension

After implementing the explainable rule-based model, the project may experimentally compare machine-learning approaches.

Possible algorithms:

- Logistic Regression
- Decision Tree
- Random Forest

Possible prediction target:

PASS / FAIL

or:

LOW / MEDIUM / HIGH RISK

Possible input features:

Attendance rate
Previous GPA
Current assessment average
Submission rate
Number of missed assessments

Recent performance trend
Learning engagement

Evaluation metrics:

- Accuracy
- Precision
- Recall
- F1 Score
- Confusion Matrix
- ROC-AUC

The machine-learning model will be treated as decision support rather than an automated decision-maker.

26. Academic Intervention Management

Lecturers or academic advisors will be able to create support interventions.

Example:

```
Student:
ST12345

Module:
Database Systems

Intervention:
Academic Consultation

Reason:
Declining assessment performance

Created:
2026-08-18

Follow-up:
2026-09-01

Status:
OPEN
```

Possible intervention types:

- Lecturer consultation

- Academic advising
- Peer tutoring
- Additional tutorial
- Revision session
- Study planning assistance

27. Intervention Outcome Analytics

UniPulse will analyze intervention outcomes.

Example:

Intervention	Students	Improvement Observed
Peer tutoring	120	78%
Extra tutorial	75	72%
Academic consultation	95	67%
Revision session	68	61%

The platform should describe these as observed relationships rather than proving that the intervention caused the improvement.

28. Student Journey Timeline

Each student can have a chronological progress history.

Example:

September
• Semester started
October
• Attendance 91%
November
△ Attendance decreased to 73%
December
△ Statistics quiz performance decreased
January

- Advisor consultation recorded

February

- ✓ Attendance increased to 84%

March

- ✓ Statistics performance improved

This allows students and advisors to understand academic development as a journey rather than isolated marks.

29. Cohort Analytics

Administrators can compare groups.

Example:

BSc Information Technology

	2022	2023	2024
Average GPA	3.01	3.12	3.19
Pass Rate	82%	85%	87%
Attendance	76%	79%	81%
Withdrawal	7%	5%	4%

Filters can include:

- Program
- Department
- Semester
- Academic year
- Module

30. Data Quality Monitoring

Since data quality is important in analytics, UniPulse will include an administrator-facing data-quality module.

Example:

Total student records	5,000
Missing attendance records	231
Invalid assessment values	17
Duplicate records	4
Incomplete enrollments	12

The ETL pipeline will check for:

- Missing values
- Duplicate rows
- Invalid grade ranges
- Invalid attendance records
- Incorrect data types
- Invalid foreign-key relationships
- Unexpected outliers

31. Business Intelligence Dashboard

In addition to React dashboards, a separate Power BI solution will be developed.

Dashboard 1 — University Overview

Contains:

- Total students
 - Average GPA
 - Pass rate
 - Attendance rate
 - Module completion
 - Academic attention statistics
 - Semester trends
-

Dashboard 2 — Academic Performance

Contains:

- Grade distributions
 - GPA trends
 - Program comparisons
 - Department comparisons
 - Module comparisons
 - Pass/fail rates
-

Dashboard 3 — Engagement

Contains:

- Attendance trends
 - Submission completion
 - Learning engagement
 - Attendance vs grades
 - Engagement by module
-

Dashboard 4 — Student Support

Contains:

- Students by attention category
 - Intervention counts
 - Intervention outcomes
 - Follow-up status
 - Support trends
-

Dashboard 5 — Module Intelligence

Contains:

- Difficult modules
 - Difficult assessments
 - Topic-level performance
 - Failure rates
 - Repeat rates
 - Module comparisons
-

32. Proposed Technology Stack

Frontend

- React
 - Next.js
 - TypeScript
 - Tailwind CSS
 - Recharts
 - Axios or Fetch API
-

Backend

- Java
 - Spring Boot
 - Spring Web
 - Spring Security
 - Spring Validation
 - Spring Data JPA
 - Hibernate
 - JWT Authentication
-

Database

- PostgreSQL
 - Flyway migrations
-

Data Analytics

- Python
 - Pandas
 - NumPy
 - Matplotlib
 - Jupyter Notebook
 - Scikit-learn
-

Business Intelligence

- Microsoft Power BI
-

Testing

Backend

- JUnit
- Mockito
- Spring Boot Test
- Testcontainers

API

- Postman

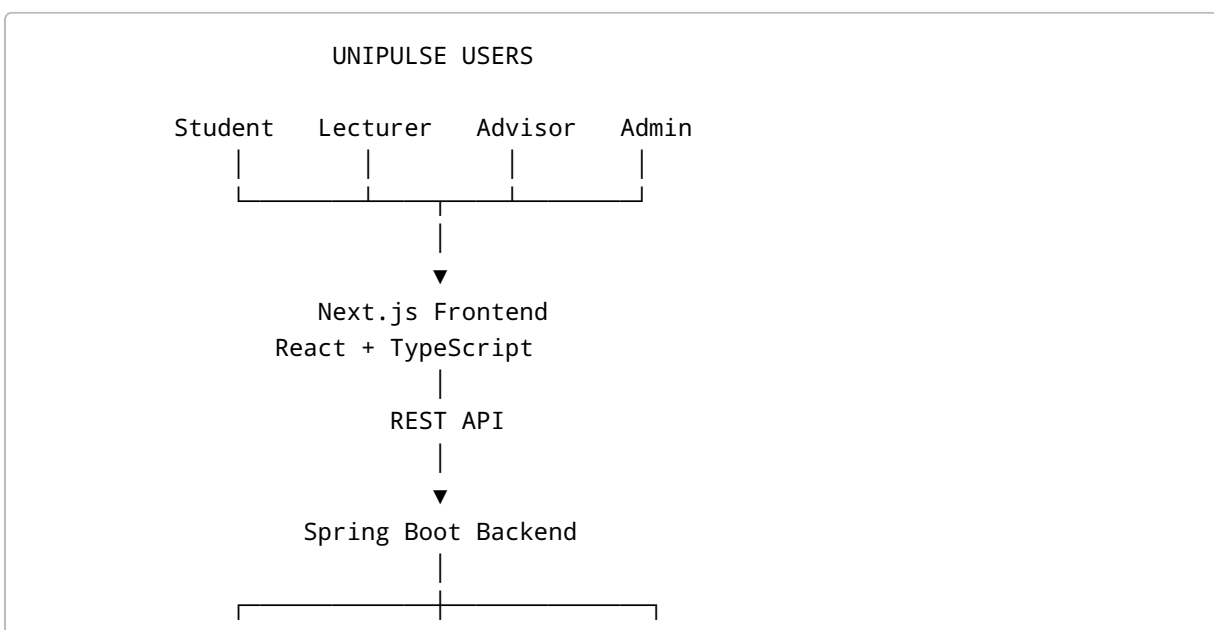
Analytics

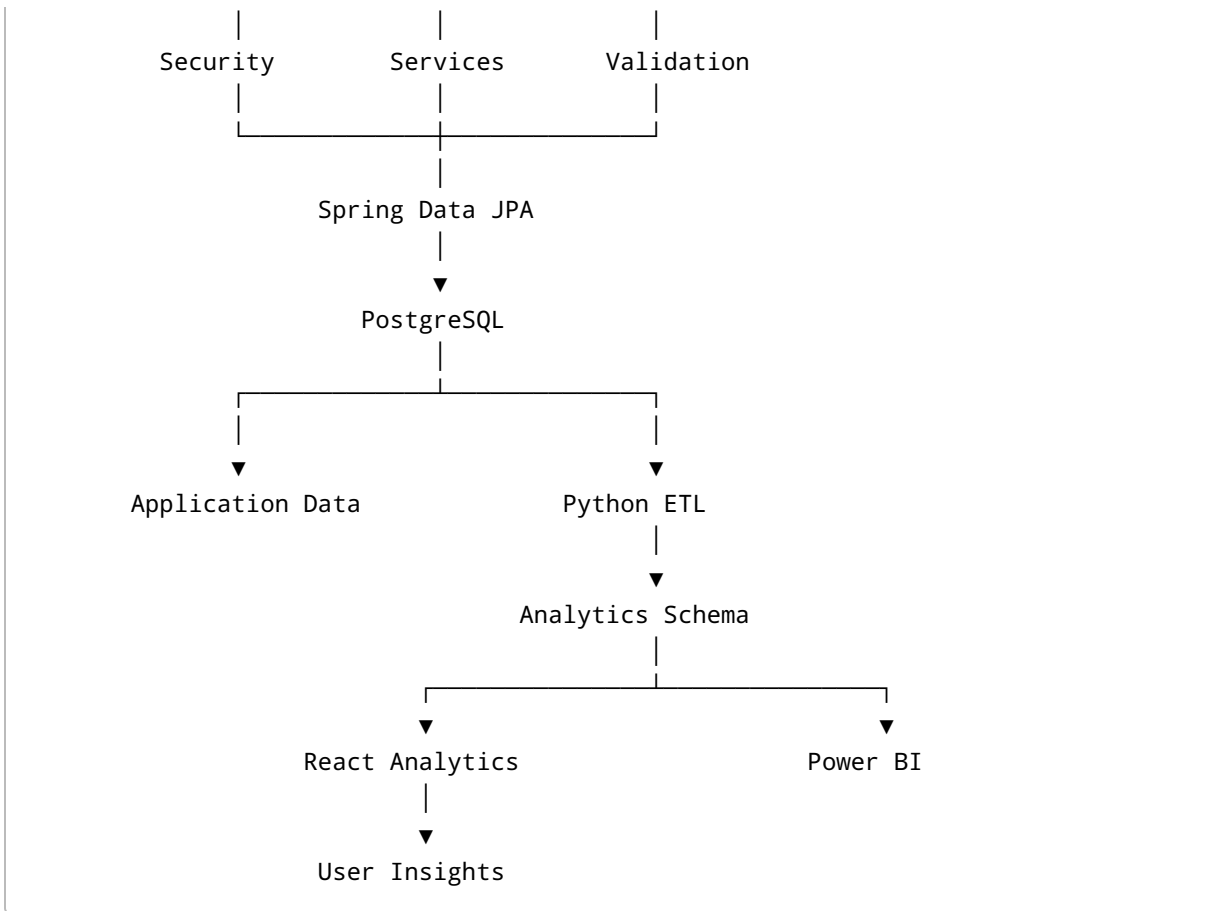
- Pytest

DevOps

- Git
 - GitHub
 - Docker
 - Docker Compose
 - GitHub Actions
-

33. Proposed System Architecture





34. Proposed Database Tables

The database may contain the following core tables.

User Management

```
users
roles
student_profiles
lecturer_profiles
advisor_profiles
```

Academic Structure

```
faculties
departments
```

programs
academic_years
semesters
modules

Enrollment

student_program_enrollments
module_enrollments

Assessments

assessments
assessment_topics
assessment_results

Attendance

attendance_sessions
attendance_records

Academic Support

attention_indicators
interventions
intervention_followups

Analytics

student_analytics
module_analytics
semester_analytics
cohort_analytics

Flexible Event Data

learning_events

35. Use of JSONB

Most academic data will be stored relationally.

For example:

```
students
modules
assessments
attendance
results
```

Semi-structured learning events can use PostgreSQL JSONB.

Example:

```
{
  "event": "resource_view",
  "resource": "lecture_04.pdf",
  "duration": 325,
  "device": "mobile"
}
```

Another event:

```
{
  "event": "quiz_attempt",
  "quizId": 42,
  "attempt": 2,
  "score": 76
}
```

Database:

```
learning_events

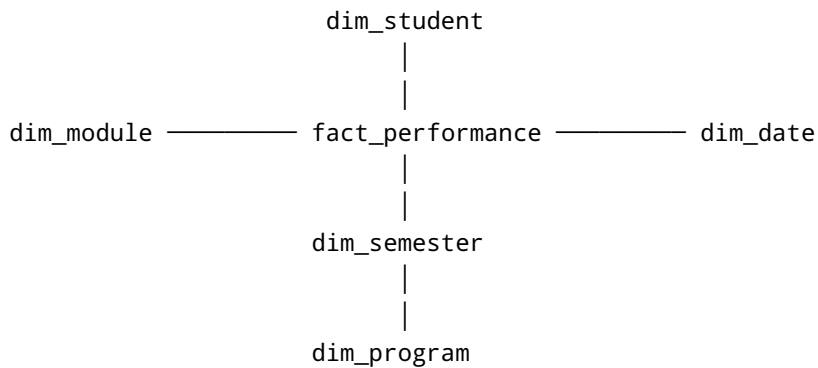
id
student_id
event_type
```

```
timestamp
payload JSONB
```

This allows UniPulse to demonstrate both relational and semi-structured data management.

36. Analytics Database Model

An analytical star schema may be developed.



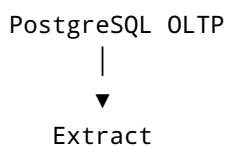
Possible fact-table measurements:

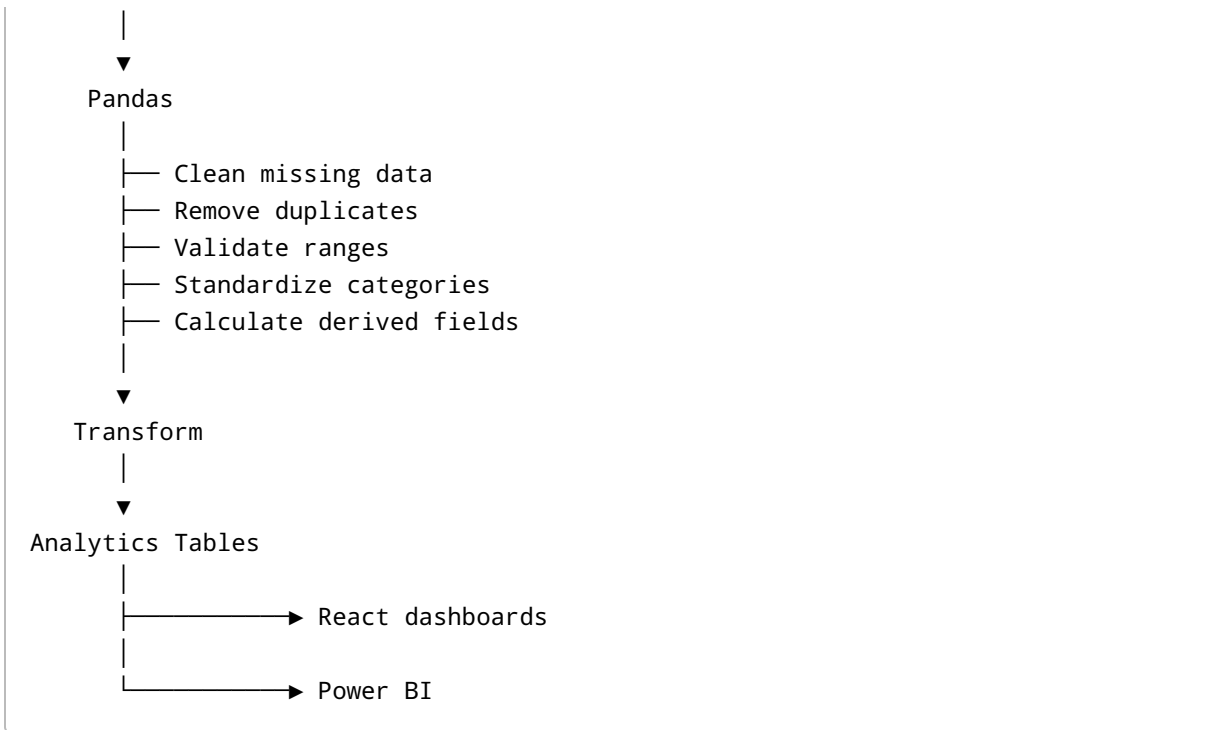
```
assessment_score
attendance_rate
submission_rate
engagement_score
final_grade
academic_health
```

This makes the project suitable for demonstrating data warehousing and BI concepts.

37. ETL Pipeline

The ETL pipeline will be implemented in Python.





38. Data Collection Strategy

Because access to real private university data may not be available, the project will primarily use synthetic academic data.

Python can generate:

5,000 students

4 faculties

8 departments

10 programs

80 modules

10 semesters

400,000 attendance records

150,000 assessment records

200,000 learning events

Libraries:

- Faker
- NumPy
- Pandas

The synthetic dataset can intentionally include:

- Missing values
- Duplicate records
- Outliers
- Varying attendance patterns
- Different academic-performance levels
- Different module difficulties
- Different engagement levels

This provides realistic data-cleaning and analytics problems.

39. Functional Requirements

The system shall allow users to:

Authentication

- Register
- Login
- Logout
- Reset password

Student

- View academic information
- View results
- View attendance
- View GPA
- View analytics
- Set grade goals
- Simulate grades
- View academic health
- View notifications

Lecturer

- Manage modules
- Create assessments
- Enter results
- Record attendance
- View class analytics
- Review student attention indicators
- Create interventions

Advisor

- Review student progress
- Review indicators
- Create intervention records
- Record follow-ups

Administrator

- Manage users
 - Manage programs
 - Manage departments
 - Manage modules
 - View institutional analytics
 - Generate reports
 - Monitor data quality
-

40. Non-Functional Requirements

Security

The system should:

- Hash passwords
 - Use JWT authentication
 - Enforce RBAC
 - Validate API input
 - Protect private student information
 - Restrict access to sensitive academic data
-

Performance

The system should:

- Use pagination for large datasets
 - Create appropriate database indexes
 - Optimize commonly used queries
 - Avoid unnecessary frontend data loading
 - Use analytical summary tables where necessary
-

Reliability

The system should:

- Validate assessment values
 - Prevent invalid module enrollments
 - Maintain database referential integrity
 - Use database transactions where necessary
-

Maintainability

The application should use:

```
Controller
  ↓
Service
  ↓
Repository
  ↓
Database
```

with clear separation of concerns.

Scalability

The architecture should allow future expansion through:

- Database indexing
- Partitioning
- Caching
- Separate analytics services

- Dedicated analytical databases
 - Cloud deployment
-

Usability

The interface should:

- Be responsive
 - Provide clear navigation
 - Use understandable dashboards
 - Explain risk indicators
 - Avoid overly technical language for students
-

41. Privacy and Ethical Considerations

Academic information is sensitive.

Therefore UniPulse will follow several principles.

Access Control

Students should only access their own information.

Lecturers should only access students associated with their modules.

Advisors should only access assigned students.

Administrators should have appropriate controlled privileges.

Explainability

The system should explain why an academic attention indicator is shown.

It should not provide unexplained predictions.

Human Oversight

Predictions should support decisions rather than automatically make decisions.

For example, a student should never automatically receive an academic penalty because a model classified them as high risk.

Sensitive Attributes

Risk models should avoid using sensitive characteristics unless there is a legitimate, carefully governed research reason.

Language

The system should say:

"Current academic patterns indicate that additional support may be useful."

rather than:

"This student will fail."

42. Testing Strategy

Unit Testing

Test individual service functions.

Examples:

GPA calculation

Grade calculation

Academic health calculation

Attendance calculation

Grade simulation

Integration Testing

Test interactions between:

```
Controller
Service
Repository
PostgreSQL
```

Testcontainers may be used to create temporary PostgreSQL instances.

API Testing

Postman will be used to test:

- Authentication
 - Students
 - Modules
 - Assessments
 - Results
 - Attendance
 - Analytics
 - Interventions
-

Data Pipeline Testing

Python analytics tests will check:

- Missing-value handling
 - Duplicate removal
 - Transformation logic
 - KPI calculations
 - Invalid-range detection
-

User Acceptance Testing

Representative scenarios may include:

```
Student checks current GPA.

Student simulates final-exam result.

Lecturer enters assessment marks.

Lecturer reviews class performance.
```

Advisor creates intervention.

Administrator compares cohorts.

43. Deployment Strategy

The system may be containerized using Docker.

Docker Compose

```
graph TD
    Root[Docker Compose] --- frontend
    Root --- backend
    Root --- database
    Root --- analytics
    frontend --- NextJS[Next.js]
    backend --- SpringBoot[Spring Boot]
    database --- PostgreSQL[PostgreSQL]
    analytics --- Python
```

Possible production deployment:

Frontend
Vercel

Backend
Render / Railway / AWS

Database
Managed PostgreSQL

BI
Power BI Service

44. Proposed Development Methodology

The project can follow an Agile iterative development approach.

Each iteration will deliver a working feature.

Example:

Sprint 1
Authentication + user management

Sprint 2
Academic structure + modules

Sprint 3
Assessments + results

Sprint 4
Attendance

Sprint 5
Student and lecturer dashboards

Sprint 6
ETL + analytics

Sprint 7
Academic intelligence

Sprint 8
Power BI + deployment

45. Development Timeline

Weeks 1-2 — Planning and Design

Deliverables:

- Problem definition
- Requirements
- Literature review
- Use-case diagrams

- System architecture
 - ER diagram
 - UI wireframes
-

Weeks 3–4 — Core Backend

Develop:

- Spring Boot project
 - PostgreSQL
 - Authentication
 - Spring Security
 - JWT
 - Users
 - Student profiles
 - Lecturer profiles
 - Program management
 - Module management
-

Weeks 5–6 — Academic Management

Develop:

- Module enrollment
 - Assessments
 - Results
 - Attendance
 - Grade calculation
 - GPA calculation
-

Weeks 7–8 — Main Frontend

Develop:

- Student dashboard
 - Lecturer dashboard
 - Advisor dashboard
 - Admin dashboard
 - Module pages
 - Assessment pages
 - Attendance pages
-

Weeks 9–10 — Analytics

Develop:

- Synthetic dataset
 - ETL pipeline
 - Data cleaning
 - Exploratory analysis
 - Analytics schema
 - Performance analytics
 - Attendance analytics
-

Weeks 11–12 — Intelligence Features

Develop:

- Academic Progress Twin
 - Academic Health Score
 - Grade Simulator
 - GPA Planner
 - Academic Attention Indicator
 - Assessment Difficulty Analysis
 - Module Difficulty Analysis
-

Week 13 — Support Analytics

Develop:

- Advisor workflow
 - Interventions
 - Follow-ups
 - Intervention outcome analytics
 - Student timeline
-

Week 14 — BI and Finalization

Develop:

- Power BI dashboards
- Automated tests
- Docker configuration
- Deployment

- Documentation
 - Final demonstrations
-

46. Expected Outcomes

At the end of the project, UniPulse should provide:

1. A complete responsive university web platform.
 2. Secure multi-role authentication.
 3. Academic data management.
 4. Student and lecturer dashboards.
 5. Attendance and assessment management.
 6. Academic-progress analytics.
 7. Academic health indicators.
 8. Grade and GPA simulation.
 9. Academic attention indicators.
 10. Advisor intervention workflows.
 11. Module and assessment analysis.
 12. Python ETL/data analytics pipeline.
 13. Analytical data model.
 14. Power BI dashboards.
 15. A deployed full-stack application.
-

47. Project Deliverables

The final project should include:

- Requirements specification
 - Use-case diagram
 - ER diagram
 - Architecture diagram
 - UI wireframes
 - Source code
 - PostgreSQL database
 - Spring Boot REST API
 - React/Next.js frontend
 - Python analytics pipeline
 - Jupyter notebooks
 - Power BI dashboard
 - API documentation
 - Test cases
 - Docker configuration
 - GitHub repository
 - Deployed application
 - Project report
 - Demonstration video
-

48. Main Technologies Demonstrated

Software Engineering

```
Java
Spring Boot
Spring Security
REST API
Spring Data JPA
Hibernate
PostgreSQL
React
Next.js
TypeScript
JWT
RBAC
Testing
Docker
```

CI/CD
Git

Data Analytics

Python
SQL
Pandas
NumPy
Data Cleaning
EDA
Statistics
Trend Analysis
KPI Design
ETL
Star Schema
Power BI
Data Visualization
Predictive Analytics

49. Project Innovation

The innovation of UniPulse does not come from one single algorithm.

It comes from integrating several academic functions into one data-driven platform.

The most distinctive features are:

1. Academic Progress Twin

A continually updated representation of each student's academic progress.

2. What-If Grade Simulator

Allows students to calculate the results required to achieve specific academic targets.

3. GPA Goal Planner

Supports semester-level academic planning.

4. Academic Attention Indicator

Identifies patterns that may indicate students require academic support.

5. Module and Assessment Difficulty Intelligence

Identifies comparatively challenging academic areas.

6. Academic Intervention Management

Allows academic advisors and lecturers to respond to identified issues.

7. Intervention Outcome Analytics

Analyzes whether student outcomes changed after support activities.

8. Dual Analytics Environment

Operational analytics are integrated into the React application, while institutional analytics are provided through Power BI.

50. Future Enhancements

After completing the main system, the following could be considered.

LMS Integration

Integrate with Moodle or another learning platform.

Email Notifications

Send reminders for:

- Low attendance
- Missing assessments
- Advisor appointments

Mobile Application

Develop a Flutter or React Native student application.

Advanced Predictive Analytics

Experiment with more advanced student-performance models.

Recommendation Engine

Recommend learning resources based on weak academic topics.

Automated Report Generation

Generate semester performance reports.

Natural Language Analytics

Allow administrators to ask:

"Which modules had the highest failure rates this semester?"

Dedicated Data Warehouse

Move large analytical workloads from PostgreSQL into a dedicated analytical environment if required.

51. Scope Control

Because UniPulse contains many potential features, the project should be divided into priorities.

Must Have — MVP

- Authentication
- Roles
- Students
- Lecturers
- Programs
- Modules
- Enrollments
- Assessments
- Results
- Attendance
- GPA
- Student dashboard
- Lecturer dashboard
- Basic analytics
- Grade simulator
- PostgreSQL database
- Python ETL
- Power BI dashboard

Should Have

- Academic Progress Twin
- Academic Health Score
- Academic Attention Indicator
- Advisor dashboard

- Interventions
- Module difficulty analysis
- Assessment difficulty analysis
- Cohort analytics

Could Have

- Machine-learning prediction
- Student segmentation
- Learning-event analytics
- Recommendation engine
- Advanced intervention analysis

This prioritization will prevent the project from becoming too large for an individual developer.

52. Success Criteria

The project can be considered successful if:

- Users can securely access role-specific functionality.
 - Academic data can be reliably stored and retrieved.
 - Lecturers can manage assessments and attendance.
 - Students can monitor progress.
 - GPA and grade calculations are correct.
 - Academic dashboards provide meaningful insights.
 - ETL successfully converts operational data into analytical data.
 - Power BI dashboards provide useful institutional reporting.
 - Academic attention indicators are transparent and reproducible.
 - Grade simulation produces mathematically correct results.
 - The system is tested and deployed successfully.
-

53. Conclusion

UniPulse proposes a modern approach to university academic management by moving beyond simple student-record storage.

The platform will combine:

academic management + full-stack software engineering + data analytics + decision support.

The Spring Boot backend will demonstrate enterprise-style backend engineering through REST APIs, Spring Security, Spring Data JPA, PostgreSQL, authentication, authorization, testing, and application architecture.

The React/Next.js frontend will provide modern dashboards and interactive academic-management interfaces.

The Python analytics environment will demonstrate data cleaning, ETL, exploratory analysis, statistical analysis, trend analysis, and optional predictive modeling.

Power BI will provide institutional business-intelligence dashboards for lecturers, administrators, and decision-makers.

Therefore, UniPulse will function not simply as a **Student Management System**, but as a:

Student Success and Academic Intelligence Platform that transforms university academic records into actionable information for students, lecturers, advisors, and university management.

This provides a strong real-world individual project that can be presented differently depending on the internship position:

For Software Engineering: emphasize application architecture, Spring Boot, React, REST APIs, PostgreSQL, security, testing, Docker, and deployment.

For Data Analytics: emphasize SQL, Python, ETL, Pandas, exploratory analysis, KPIs, data modeling, Power BI, trends, and academic-performance analytics.

As a result, a single UniPulse project can provide strong evidence of practical capability in both **Full-Stack Software Engineering and Data Analytics**.